School of Future Tech

Case Study Report

on

Restaurant Order Management System

by

Name : Aaryan Nilesh Keni

150096725197

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

- Order management and billing are critical day-to-day operations for any dining establishment.
- However, manual paper-based ordering systems can be slow, prone to human error, and difficult to modify once an order is placed.
- This case study focuses on designing and implementing a digital *Restaurant Order Management System* that automatically handles menu display, order placement, order modification, and final bill generation using Object-Oriented Programming (OOP) in C++.
- The case study demonstrates how real-world entities (like menus and orders) can be represented as data structures and manipulated using class methods.
- It highlights the importance of modular programming, dynamic memory management (via vectors), and user-friendly console interfaces in building a practical and reliable software system.

2. Problem Statement / Case Background (Abstract)

Background

- Traditional pen-and-paper order taking is not sufficient to handle high-traffic restaurant environments efficiently, often leading to calculation errors or mismanaged orders.
- Software-based systems can dynamically store items, allow seamless modifications, and instantly calculate totals with 100% accuracy.

Abstract

- This case study presents the design and implementation of an **Restaurant Order Management System** using C++.
- A predefined menu is initialized with items (Burger, Pizza, Pasta, Cold Coffee) and their respective prices.
- The system uses dynamic arrays (`std::vector`) to store the user's current order, allowing them to add or remove items on the fly.
- The implementation is divided into modular files (Header files for declarations, and CPP files for logic and execution) to maintain clean code architecture.
- The system is evaluated through a console-driven menu that successfully accepts user inputs, modifies state, and prints a formatted final bill.

3. Case Study Design

The Restaurant Order Management System is designed as a modular pipeline with the following stages:

Data Modeling

- Use a `MenuItem` structure to bind an item's name (`std::string`) and price (`double`) together.
- Use a `Restaurant` class to encapsulate the state (the menu and the current order).

System Operations & Flow

- **Initialization:** Pre-load the menu vector with standard items and prices.
- **Display:** Output the menu dynamically via a loop so the user can see available item numbers.
- **Order Placement:** Continuously accept integer inputs from the user to add corresponding items to the `currentOrder` vector until the user inputs '0'.
- **Order Modification:** Display the current cart and allow the user to input an index to remove a specific item from the vector.
- **Bill Generation:** Iterate through the `currentOrder` vector, sum the prices, output a formatted receipt, and clear the cart for the next customer.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms:

- **Object-Oriented Programming (OOP):** Encapsulation of data and methods within the `Restaurant` class.
- **Dynamic Data Structures:** Using `std::vector` for dynamic array resizing when adding/removing order items.
- **Control Flow Algorithms:** * `do-while` loops for the main interactive application menu.
    - `switch-case` statements for routing user choices to specific class methods.
    - `while` loops for continuous input reading during order placement.

- **Data Deletion Algorithms:** Using the `.erase()` method combined with iterators to remove specific elements from the middle of a vector.

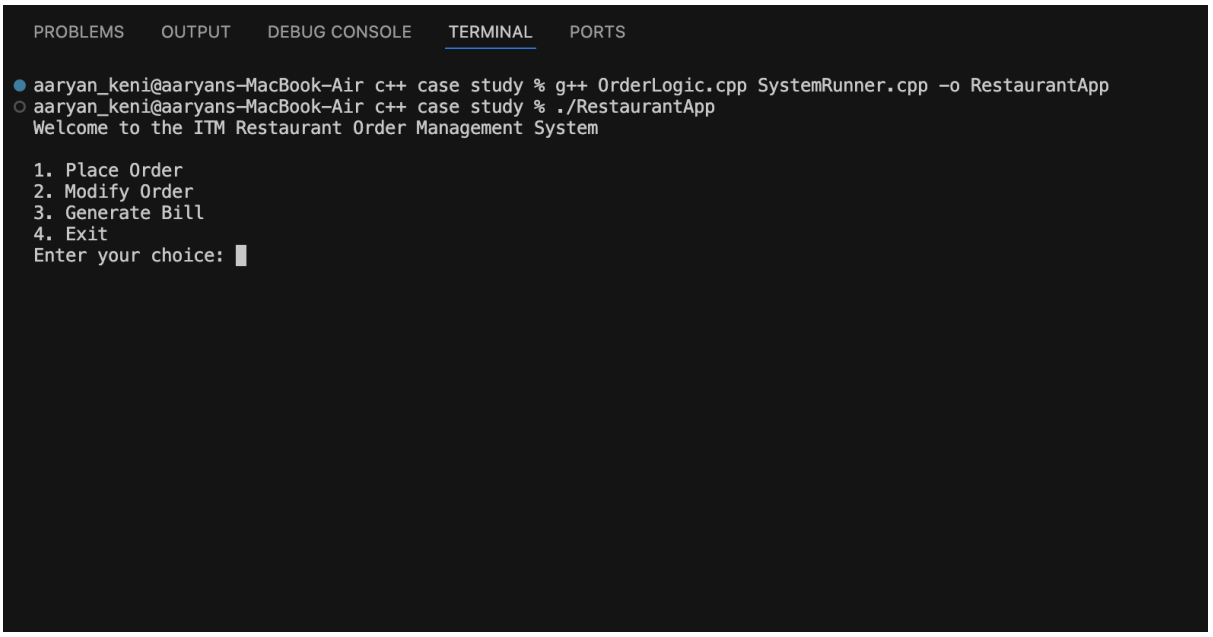
Technology Stack Used for the Case

- **Programming language:** C++
- **Standard Libraries:** * `<iostream>` for console input/output streams.
    - `<string>` for text handling.
    - `<vector>` for dynamic data storage.
- **Environment:** Any C++ IDE (VS Code, Code::Blocks, Dev-C++) or Terminal with GCC/Clang compiler.
- **Architecture:** Multi-file compilation (`.h` and `.cpp` separation).

5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation is organized in separate C++ files to ensure modularity:

- **`MenuManager.h`**
    - Defines the *`MenuItem`* struct.
    - Declares the *`Restaurant`* class, its private vectors (*`menu`* and `currentOrder`), and public method signatures (showMenu, `placeOrder`, `modifyOrder`, `generateBill`).



- **`OrderLogic.cpp`**
    - Implements the logic for all class methods. Handles the initialization of the hardcoded menu, pushing items to the order vector, erasing items, and mathematically calculating the total bill.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

aaryan_keni@aaryans-MacBook-Air c++ case study % ./RestaurantApp
Welcome to the ITM Restaurant Order Management System

1. Place Order
2. Modify Order
3. Generate Bill
4. Exit
Enter your choice: 1

--- RESTAURANT MENU ---
1. Burger - Rs.150
2. Pizza - Rs.450
3. Pasta - Rs.300
4. Cold Coffee - Rs.180

Enter Item Number to add to your order (Press 0 to finish): 1 1 2 2 3 4 3 4
>> Burger added.
Add another item? (Press 0 to finish): >> Burger added.
Add another item? (Press 0 to finish): >> Pizza added.
Add another item? (Press 0 to finish): >> Pizza added.
Add another item? (Press 0 to finish): >> Pasta added.
Add another item? (Press 0 to finish): >> Cold Coffee added.
Add another item? (Press 0 to finish): >> Pasta added.
Add another item? (Press 0 to finish): >> Cold Coffee added.
Add another item? (Press 0 to finish): 0

Order Placed Successfully

1. Place Order
2. Modify Order
3. Generate Bill
4. Exit
Enter your choice: 3

========================
        FINAL BILL
========================
Burger           Rs.150
Burger           Rs.150
Pizza            Rs.450
Pizza            Rs.450
Pasta            Rs.300
Cold Coffee              Rs.180
Pasta            Rs.300
Cold Coffee              Rs.180
------------------------
Total Bill:      Rs.2160
========================
```

- **`SystemRunner.cpp`**
    - Contains the `main()` function. Acts as the entry point and provides a simple command-line interface (CLI) where the user enters choices (1 to 4) to interact with the system.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

aaryan_keni@aaryans-MacBook-Air c++ case study % ./RestaurantApp
1. Place Order
2. Modify Order
3. Generate Bill
4. Exit
Enter your choice: 2

--- Your Current Order ---
1. Burger (Rs.150)
2. Burger (Rs.150)
3. Burger (Rs.150)
4. Pizza (Rs.450)
5. Pizza (Rs.450)
6. Pasta (Rs.300)
7. Pasta (Rs.300)
Enter Item Number to remove from order (Press 0 to cancel): 1 1
Item removed successfully.

1. Place Order
2. Modify Order
3. Generate Bill
4. Exit
Enter your choice:
--- RESTAURANT MENU ---
1. Burger - Rs.150
2. Pizza - Rs.450
3. Pasta - Rs.300
4. Cold Coffee - Rs.180

Enter Item Number to add to your order (Press 0 to finish): 0

Order Placed Successfully

1. Place Order
2. Modify Order
3. Generate Bill
4. Exit
Enter your choice: 3

========================
       FINAL BILL
========================
Burger           Rs.150
Burger           Rs.150
Pizza            Rs.450
Pizza            Rs.450
Pasta            Rs.300
Pasta            Rs.300
------------------------
Total Bill:      Rs.1800
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

- Utilizing `std::vector` provides a highly flexible way to manage unpredictable order sizes compared to static arrays.
- Separating class declarations into header files and logic into CPP files significantly improves code readability and maintainability.
- Input validation (checking if the user's choice is within the bounds of the vector size) prevents system crashes and out-of-bounds errors.

Conclusion

- This case study demonstrates a complete pipeline for building a functional Restaurant Order Management System using C++.
- It integrates core programming concepts like struct modeling, OOP, standard template library (STL) usage, and user-interactive loops into a coherent system, providing a strong foundation for more advanced software development.

7. References

- Official C++ Documentation for Standard Template Library (STL) Vectors (`std::vector`).
- C++ `<iostream>` and `<string>` library documentation for console I/O and text manipulation.
- Academic tutorials on Object-Oriented Programming principles in C++.